

1. Описание класса в C++. Поля и методы. Внутренние и внешние методы. Разграничение доступа к элементам класса. Спецификаторы доступа, set- и get- методы. Константные методы

Класс — это пользовательский тип данных, который объединяет в себе данные (поля/свойства) и функции (методы) для работы с этими данными. Это основа объектно-ориентированного программирования (ООП), реализующая принцип инкапсуляции (сокрытия данных).

Поля (данные-члены) — это переменные, объявленные внутри класса. Они описывают состояние объекта.

Методы (функции-члены) — это функции, объявленные внутри класса. Они описывают поведение объекта (операции над полями).

Внутренние и внешние методы

Внутренние (inline) методы: Определяются внутри тела класса (как `displayInfo` в примере выше). Компилятор старается подставить их код непосредственно в место вызова (сделать `inline`), что может повысить производительность для простых методов.

Внешние методы: Объявляются только внутри класса, а их определение находится снаружи, с использованием оператора разрешения области видимости `::`. Это улучшает читаемость кода и разделяет интерфейс (заголовочный файл `.h`) и реализацию (файл `.cpp`).

Разграничение доступа. Спецификаторы доступа

Это ключевой механизм инкапсуляции. Он управляет видимостью членов класса извне.

private (закрытый): Члены доступны только методам этого же класса. Поля данных почти всегда объявляют `private`. Это защищает внутреннее состояние объекта от некорректного изменения.

protected (защищённый): Как `private`, но также доступны классам-наследникам (при наследовании).

public (открытый): Члены доступны из любого места программы (из других функций, классов). Методы для взаимодействия с объектом обычно `public`.

По умолчанию, в классе (`class`) все члены являются `private`, а в структуре (`struct`) — `public`.

set- и get-методы (мутаторы и аксессоры)

Поскольку поля обычно приватные, для контролируемого доступа к ним создают публичные методы.

set-метод (мутатор): Позволяет безопасно установить значение поля, добавив проверку.

get-метод (аксессор): Позволяет безопасно получить значение поля (часто как копию).

Константные методы

Это методы, которые не изменяют состояние объекта (не меняют его поля). Ключевое слово **const** ставится после списка параметров метода.

Позволяют компилятору гарантировать, что метод не изменит объект.

Могут быть вызваны у константных объектов (`const MyClass obj;`).

get-методы почти всегда должны быть константными.

2.Конструкторы. Списки инициализаторов в конструкторах.

Конструктор по умолчанию. Конструктор копирования.

Конструктор — это специальный метод класса, который автоматически вызывается при создании объекта. Его задачи:

Выделение памяти под объект (автоматически).

Инициализация полей объекта.

Установка объекта в безопасное начальное состояние.

Имя конструктора: Совпадает с именем класса.

1. Конструктор по умолчанию (Default Constructor)

Не принимает аргументов.

Автоматически генерируется компилятором, если вы не объявили ни одного конструктора.

Для встроенных типов (`int`, `double`, указатели) поля не инициализируются (в них "мусор").

Для полей-объектов классов вызываются их конструкторы по умолчанию.

2. Параметризованный конструктор

Принимает аргументы для инициализации полей.

3. Конструктор копирования (Copy Constructor)

Принимает константную ссылку на объект того же класса (`const MyClass &`).

Создаёт новый объект как копию существующего.

Вызывается в ситуациях:

Передача объекта в функцию по значению.

Возвращение объекта из функции по значению (может быть оптимизировано (RVO/NRVO)).

Явное создание копии: Point p2(p1);

Конструктор перемещения (Move Constructor)

Что это?

Конструктор перемещения — это специальный конструктор класса, который создаёт новый объект, "забирая" (перемещая) ресурсы у существующего объекта-источника, который вскоре будет уничтожен.

Конструктор перемещения - для эффективной передачи ресурсов

Он "крадёт" ресурсы у источника, оставляя его в валидном, но опустошённом состоянии. Ключевые слова: &&, noexcept, std::move(), правило пяти.

Списки инициализации в конструкторах

Список инициализации — это специальный синтаксис в C++, позволяющий инициализировать поля класса до выполнения тела конструктора. Он располагается после двоеточия : и перед телом конструктора.

Зачем нужны списки инициализации?

1. Единственный способ инициализировать определенные типы полей

Для этих типов полей список инициализации — обязателен:

a) const поля

b) Ссылки (&)

c) Поля-объекты без конструктора по умолчанию

Список инициализации — это часть конструктора после :

Обязателен для: const полей, ссылок, полей без конструктора по умолчанию

Повышает производительность: вызывает конструкторы вместо конструктора по умолчанию + присваивания

Порядок инициализации определяется порядком объявления в классе

Лучшая практика: всегда использовать список инициализации для всех полей

В общем, основная разница в эффективности, нежели инициализировать в теле.

3. Деструкторы. Деструктор по умолчанию Необходимость написания собственных деструкторов.

Деструктор — это специальный метод класса, который автоматически вызывается при уничтожении объекта. Его основная задача — освобождение ресурсов, которые объект захватил в течение своей жизни.

Имя: ~ + имя класса

Не принимает параметров (в отличие от конструкторов)

Не возвращает значение (даже void)

Не может быть перегружен (только один деструктор на класс)

Не может быть const, volatile, static

Необходимость написания собственных деструкторов

Когда деструктор по умолчанию НЕДОСТАТОЧЕН?

Правило: Если класс управляет ресурсами, он должен иметь собственный деструктор для их освобождения.

Что такое деструктор по умолчанию?

Деструктор по умолчанию — это деструктор, который автоматически генерируется компилятором, если вы не объявляете собственный деструктор в классе.

Деструктор по умолчанию генерируется автоматически, если нет пользовательского

= default — явная инструкция компилятору "сгенерируй деструктор по умолчанию"

Пустой деструктор {} ≠ деструктор по умолчанию — это пользовательский деструктор!

Тривиальность важна для оптимизаций — = default её сохраняет, {} — ломает.

4. Дружественные функции и классы.

Что такое дружественность?

В C++ механизмы инкапсуляции (`private` и `protected`) защищают данные класса от прямого доступа извне. Однако иногда возникают ситуации, когда внешней функции или другому классу необходимо получить доступ к скрытым полям для эффективной работы.

Дружественность — это механизм, позволяющий функции или классу получить полный доступ ко всем членам другого класса (включая `private` и `protected`), не являясь при этом его членом.

1. Дружественные функции (Friend Functions)

Дружественная функция — это функция, которая не является членом класса, но имеет право обращаться к его закрытым компонентам.

Основные особенности:

Она не является методом класса (не принадлежит ему).

Она объявляется внутри класса с ключевым словом `friend`.

Поскольку это не метод, у нее нет указателя `this`. Объект класса должен передаваться ей в качестве аргумента.

Ее можно объявить в любом разделе класса (`public`, `private` или `protected`) — это не влияет на ее права.

2. Дружественные классы (Friend Classes)

Если мы объявляем целый класс В другом классе А, это означает, что все методы класса В получают доступ ко всем закрытым членам класса А.

3. Важные свойства дружественности (Золотые правила для зачета)

На этих нюансах часто "ловят" преподаватели. Запомни их:

Дружественность не является взаимной: Если класс А объявил класс В своим другом, это не значит, что класс А автоматически стал другом класса В.

Дружественность не наследуется: Если у базового класса есть "друг", то для производного (наследника) этот "друг" не будет иметь доступа к приватным полям, если это не указано явно. И наоборот: "друзья" наследника не имеют доступа к приватным полям родителя.

Дружественность не является транзитивной: Если А — друг В, а В — друг С, то А не является другом С.

Дружественность нельзя "взять", её можно только "отдать": Только сам класс может решить, кого объявить своим другом. Вы не можете "взломать" класс, объявив свою функцию его другом снаружи.

5. Выброс исключений в методах класса. Выбросы исключений в конструкторах. Использование стандартного класса exception для выброса исключений.

Исключения в методах работают так же, как и в обычных функциях. Когда метод встречает ошибку, которую он не может обработать на месте, он выбрасывает исключение с помощью ключевого слова `throw`.

Спецификатор `noexcept`: Если ты уверен, что метод не должен выбрасывать исключения, его помечают `noexcept`.

Выброс исключений в конструкторах

Это "звездный час" исключений в C++, но здесь кроется важная ловушка.

Основное правило:

Если конструктор выбрасывает исключение, считается, что объект не был создан.

Из этого вытекают два критических следствия:

Деструктор для этого объекта вызван не будет. C++ не вызывает деструкторы для объектов, чье создание не завершилось.

Утечки ресурсов: Если внутри конструктора ты выделил память через `new` (сырой указатель) до того, как случилось исключение, эта память утечет, так как деструктор не сработает и не очистит её.

Использование стандартного класса `std::exception`.

6. Перегрузка операторов. Способы перегрузки операторов для собственных классов.

Что такое перегрузка операторов?

Это придание стандартным операторам (+, -, *, ==, << и др.) нового смысла для работы с объектами ваших классов.

Важные ограничения:

Нельзя создавать новые операторы (например, нельзя придумать оператор `**`).

Нельзя менять приоритет и ассоциативность (умножение всегда будет выполняться раньше сложения).

Нельзя перегружать операторы для встроенных типов (нельзя переопределить `int + int`).

Способы перегрузки операторов

Существует два основных способа «научить» класс работать с операторами. Выбор зависит от того, что стоит слева от оператора.

1. Перегрузка через метод класса (Member Function)

```
class Vector {  
public:  
    double x, y;  
    Vector(double x, double y) : x(x), y(y) {}  
  
    // Перегрузка бинарного +  
    // v1 + v2 преобразуется в v1.operator+(v2)  
    Vector operator+(const Vector& other) const {  
        return Vector(this->x + other.x, this->y + other.y);  
    }  
};
```

В этом случае оператор вызывается от имени левого операнда.

Левый операнд: Всегда текущий объект (*this).

Аргументы: На один меньше, чем количество операндов (для бинарного оператора + нужен 1 аргумент — правый операнд)

2. Перегрузка через внешнюю (дружественную) функцию

```
class Vector {  
    // ... поля x и y ...  
    friend Vector operator+(double scalar, const Vector& v);  
};  
  
// Реализация вне класса  
Vector operator+(double scalar, const Vector& v) {  
    return Vector(v.x + scalar, v.y + scalar);  
}
```

Используется, если левым операндом является объект другого класса или встроенный тип (например, `double + Vector` или `cout << Vector`).

Категории операторов и нюансы

Унарные операторы (++ , -- , - , !)

Интересный момент с инкрементом/декрементом: как отличить ++x (префиксный) от x++ (постфиксный)?

Префиксный: Vector& operator++() — возвращает ссылку.

Постфиксный: Vector operator++(int) — принимает фиктивный параметр int и возвращает копию значения до изменения.

Операторы ввода-вывода (<<, >>)

Их всегда перегружают как внешние дружественные функции, потому что левый операнд — это поток (std::ostream или std::istream), а не ваш класс.

friend std::ostream& operator<<(std::ostream& os, const Vector& v) {

os << "(" << v.x << ", " << v.y << ")";

return os; // Возвращаем поток для цепочки вызовов: cout << v1 << v2;

}

Операторы, которые ОБЯЗАТЕЛЬНО должны быть методами класса:

C++ запрещает перегружать их вне класса:

= (Присваивание)

[] (Индексация)

() (Вызов функции)

-> (Доступ по указателю)

Какие операторы перегружать НЕЛЬЗЯ?

Этих «счастливчиков» всего пять, их часто спрашивают на зачете:

. (доступ к члену класса)

.* (разыменованье указателя на член класса)

:: (разрешение области видимости)

?: (тернарный оператор)

sizeof, typeid

7. Шаблоны классов. Специализация шаблонов. Вариативные шаблоны.

Шаблон класса — это «чертеж» или инструкция для компилятора, как создать класс для конкретного типа данных. Код генерируется не в момент

написания, а в момент инстанцирования (когда вы создаете объект с конкретным типом).

Ключевые моменты:

T — это параметр шаблона (placeholder).

Весь код шаблонов обычно должен находиться в заголовочных файлах (.h), так как компилятору нужно видеть все определение для генерации кода.

2. Специализация шаблонов

Иногда общая логика шаблона не подходит для конкретного типа (например, для указателей или bool). В этом случае мы создаем специальную версию шаблона.

Полная специализация

Мы полностью переписываем класс для конкретного типа.

// Общий шаблон

```
template <typename T>
```

```
class Printer {
```

```
public:
```

```
    void print(T val) { std::cout << "Value: " << val << std::endl; }
```

```
};
```

// Полная специализация для bool (хотим выводить true/false текстом)

```
template <>
```

```
class Printer<bool> {
```

```
public:
```

```
    void print(bool val) { std::cout << (val ? "Yes" : "No") << std::endl; }
```

```
};
```

3. Вариативные шаблоны (Variadic Templates)

3. Вариативные шаблоны (Variadic Templates)

Появились в C++11. Они позволяют принимать произвольное количество аргументов разных типов. Используют «многоточие» (...), которое называется пакетом параметров (parameter pack).

```
template <typename... Args>
```

```
void printAll(Args... args) {
```

```
    // В современном C++17 это делается через fold expressions:
```

```
    (std::cout << ... << args) << std::endl;
```

```
}
```

8. Полиморфизм. Статический и динамический полиморфизм.

Полиморфизм — это одна из «трех основ» ООП (вместе с инкапсуляцией и наследованием). Если говорить просто, это способность одного и того же кода работать с разными типами данных.

В C++ полиморфизм делится на два больших лагеря: тот, что работает во время компиляции, и тот, что работает во время выполнения программы.

1. Статический полиморфизм (Compile-time)

Основные механизмы:

Перегрузка функций (Overloading): Несколько функций с одним именем, но разными аргументами.

Шаблоны (Templates): Один и тот же код генерируется для разных типов (мы это разбирали в прошлом вопросе).

2. Динамический полиморфизм (Runtime)

Это то, что обычно имеют в виду, когда говорят «полиморфизм» в ООП. Здесь решение о том, какой метод вызвать, принимается в момент работы программы.

Как это работает:

У вас есть базовый класс с виртуальной функцией (virtual).

Есть производные классы, которые переопределяют (override) эту функцию.

Вы используете указатель или ссылку на базовый класс, чтобы вызвать метод.

```
class Animal {
```

```
public:
```

```
    virtual void say() { std::cout << "???" << std::endl; } // Виртуальная функция
```

```
};
```

```
class Dog : public Animal {  
public:  
    void say() override { std::cout << "Гав!" << std::endl; }  
};
```

```
class Cat : public Animal {  
public:  
    void say() override { std::cout << "Мяу!" << std::endl; }  
};
```

```
void makeItSpeak(Animal* a) {  
    a->say(); // Программа сама поймет, кто перед ней — пес или кот  
}
```

Важные «фишки» для ответа:

Зачем нужен virtual деструктор? Если в классе есть хотя бы одна виртуальная функция, деструктор обязательно должен быть виртуальным. Иначе при удалении объекта через указатель на базовый класс (`Animal* a = new Dog(); delete a;`) вызовется только деструктор родителя, и произойдет утечка памяти (хвост собаки останется в памяти!).

Чисто виртуальные функции и абстрактные классы: Если написать `virtual void say() = 0;`, функция становится «чисто виртуальной». Класс превращается в абстрактный — вы не сможете создать его объект, только использовать как интерфейс для наследников.

Ключевое слово `override`: Всегда пиши его у наследников. Оно не обязательно, но если ты ошибешься в сигнатуре (например, напишешь `Say()` вместо `say()`), компилятор выдаст ошибку, а не просто создаст новую функцию.

Механизм VTable (Таблица виртуальных методов)

Как программа понимает, какой метод вызвать? Для этого C++ использует скрытые таблицы:

Для каждого класса с виртуальными функциями создается VTable (массив адресов функций).

В каждом объекте такого класса есть скрытый указатель `vpitr`, который указывает на эту таблицу.

9. lvalue и rvalue ссылки. Конструктор перемещения, присваивание с перемещением.

1. lvalue и rvalue: В чем разница?

Прежде чем говорить о ссылках, нужно понять, что это за «звери».

lvalue (left value): Это объект, у которого есть имя и адрес в памяти. Он «живет» долго.

Пример: переменная `int x = 5;`. Здесь `x` — это lvalue.

rvalue (right value): Это временное значение, у которого нет имени и которое исчезнет сразу после использования.

Пример: число 5, результат выражения $(a + b)$ или временный объект, возвращенный из функции.

2. Ссылки: & vs &&

lvalue-ссылка (T&): Обычная ссылка, к которой мы привыкли. Она привязывается только к именованным объектам (lvalue).

```
int x = 10;
```

```
int& ref = x; // Ок
```

```
// int& ref2 = 10; // Ошибка! Нельзя привязать ссылку к временному числу
```

rvalue-ссылка (T&&): Новинка C++11. Она привязывается только к временным объектам (rvalue). Это дает нам сигнал: «Этот объект скоро умрет, мы можем забрать его ресурсы!».

```
int&& rref = 10; // Ок
```

```
// int&& rref2 = x; // Ошибка! x — это lvalue
```

3. Семантика перемещения (Move Semantics)

Представь, что у тебя есть класс «Квартира», в которой лежит куча тяжелой мебели (данных).

Копирование: Ты строишь точно такую же квартиру и покупаешь туда такую же мебель. Старая квартира остается. Это долго и дорого.

Перемещение: Ты просто забираешь ключи от старой квартиры и перевешиваешь на них бирку с новым номером. Старая квартира остается пустой. Это мгновенно.

Конструктор перемещения

Он принимает rvalue-ссылку и просто «ворует» указатель на данные у другого объекта.

```
class Buffer {  
    int* data;  
    size_t size;  
public:  
    // Конструктор перемещения  
    Buffer(Buffer&& other) noexcept {  
        this->data = other.data; // Забираем адрес памяти  
        this->size = other.size;  
  
        other.data = nullptr; // ОБЯЗАТЕЛЬНО обнуляем у старого,  
        other.size = 0; // чтобы его деструктор не удалил наши данные  
    }  
};
```

Присваивание с перемещением

Работает так же, но нужно сначала очистить свои старые данные.

```
Buffer& operator=(Buffer&& other) noexcept {  
    if (this != &other) { // Проверка на самоприсваивание  
        delete[] data; // Удаляем свое старое  
  
        data = other.data; // Крадем чужое  
        size = other.size;  
  
        other.data = nullptr; // Обнуляем чужое  
        other.size = 0;  
    }
```

```
}  
  
    return *this;  
  
}
```

4. Функция `std::move`

Важный момент: `std::move` на самом деле ничего не перемещает. Она просто делает `cast` (преобразование): превращает `lvalue` в `rvalue`. Это способ сказать компилятору: «Я знаю, что эта переменная мне больше не нужна в этом виде, разрешаю её обрабить (переместить из неё данные)».

10. Наследование. Спецификаторы доступа при наследовании. Виртуальное наследование.

1. Наследование (Inheritance)

Наследование позволяет создать новый класс (производный) на основе уже существующего (базового). Производный класс заимствует свойства и методы родителя, что позволяет избегать дублирования кода.

2. Спецификаторы доступа при наследовании

Это самый коварный момент. Спецификатор перед именем родительского класса (`public`, `protected` или `private`) определяет «верхнюю границу» доступа к его членам в наследнике.

Золотое правило:

Закрытые (`private`) члены базового класса НИКОГДА не доступны напрямую в производном классе. К ним можно обратиться только через открытые методы родителя.ежать дублирования кода.

3. Виртуальное наследование (Virtual Inheritance)

Оно нужно для решения «Ромбовидного наследования» (Diamond Problem).

Проблема:

Представь, что классы `B` и `C` наследуют класс `A`. А класс `D` наследует и `B`, и `C` одновременно. Без виртуального наследования объект класса `D` будет содержать две копии данных класса `A` (одну от `B`, другую от `C`). Это вызывает двусмысленность: какую копию переменной использовать?

Если мы укажем ключевое слово `virtual` при наследовании от самого верхнего класса, то компилятор гарантирует, что в конечном объекте будет только одна копия общего предка.

```
class A { public: int value; };
```

```
// Используем virtual
class B : virtual public A { };
class C : virtual public A { };
class D : public B, public C { };

// Теперь в классе D только одна переменная value!
```

11. Наследование и конструктор. Наследование и деструктор. Наследование и статические члены класса.

Тело конструктора наследника начинает выполняться только тогда, когда конструктор родителя полностью завершил свою работу.

Порядок вызова:

Когда ты создаешь объект наследника, конструкторы вызываются строго по цепочке: от родителя к ребенку. Сначала строится «фундамент» (база), а потом надстраиваются этажи (наследник).

Как вызвать конструктор родителя? Это делается только в списке инициализации конструктора наследника.

```
class Base {
public:
    Base(int x) { std::cout << "Base constructor: " << x << std::endl; }
};

class Derived : public Base {
public:
    // Явный вызов конструктора родителя
    Derived(int x, int y) : Base(x) {
        std::cout << "Derived constructor: " << y << std::endl;
    }
};
```

2. Наследование и деструкторы

Здесь всё работает с точностью до наоборот: Деструкторы вызываются в обратном порядке (от ребенка к родителю). Сначала разбираются «этажи», потом «фундамент».

Критически важный момент: Виртуальный деструктор

Если ты используешь полиморфизм (указатель на базовый класс указывает на объект-наследник), деструктор базового класса обязан быть виртуальным.

```
Base* obj = new Derived();
```

```
delete obj; // Если деструктор Base не virtual, вызовется ТОЛЬКО ~Base().
```

```
// Память, выделенная в Derived, утечет!
```

Если деструктор virtual, программа заглянет в таблицу виртуальных методов (VTable), найдет там деструктор Derived, выполнит его, а потом автоматически вызовет деструктор Base.

Виртуальный деструктор нужен ТОЛЬКО тогда, когда мы используем указатель на базовый класс, чтобы управлять объектом-наследником.

Сценарий «Беда» (Без virtual):

```
Base* ptr = new Derived(); // Указатель типа Base, но объект — Derived
```

```
delete ptr;
```

Если в классе Base деструктор не помечен словом virtual, случится следующее: Компилятор посмотрит на тип указателя (Base*) и скажет: «Ага, это базовый класс, вызову-ка я только деструктор ~Base()».

Итог: Деструктор ~Derived() вообще не вызовется. Всё, что создал наследник (выделенная память, открытые файлы), останется «висеть» в системе. Это утечка памяти.

3. Наследование и статические члены класса

Статические члены (static) — это переменные или методы, которые принадлежат классу вообще, а не конкретному объекту.

Как они ведут себя при наследовании?

Они наследуются: Наследник имеет к ним доступ (если позволяет спецификатор — public или protected).

Они общие (Shared): И это самое важное. Статическая переменная существует в единственном экземпляре на всю иерархию классов.

Шпаргалка для зачета:

Конструкторы: Сначала база, потом наследник. Передаем параметры через список инициализации : Base(args).

Деструкторы: Сначала наследник, потом база. Обязательно делать деструктор базового класса virtual, если есть хоть одна виртуальная функция.

Статика: Наследуется, но не копируется. Весь «род» классов пользуется одной и той же переменной в памяти.

12. Преобразования вверх и вниз по иерархии наследования.

1. Upcasting (Преобразование вверх)

Upcasting — это приведение указателя или ссылки производного класса (Derived) к типу базового класса (Base).

Особенности:

Безопасно на 100%: Поскольку «Собака» — это всегда «Животное», компилятор уверен в успехе.

Зачем нужно? Это основа полиморфизма. Позволяет писать функции, которые принимают `Animal*` и работают с любым наследником (Dog, Cat, Bird).

Dog myDog;

Animal* animalPtr = &myDog; // Upcasting - неявно и безопасно

2.Downcasting (Преобразование вниз)

Downcasting — это приведение указателя или ссылки базового класса к типу производного класса (из `Base*` в `Derived*`).

Особенности:

Опасно: Не каждое «Животное» является «Собакой». Это может быть «Кот». Если мы заставим «Кота» выполнять метод `bark()` (лаять), программа упадет.

Только явно: Компилятор заставит тебя использовать операторы приведения типов.

Два способа сделать Downcasting:

A) static_cast (Рискованный)

Компилятор просто «верит тебе на слово». Он не проверяет во время работы программы, какой объект там лежит на самом деле.

Плюс: Работает мгновенно (быстро).

Минус: Если ты ошибся и там был не тот объект, ты получишь неопределенное поведение (UB).

Б) dynamic_cast (Безопасный — рекомендуется)

Он проверяет тип объекта прямо во время выполнения программы (RTTI — Run-Time Type Information).

Как работает: Если преобразование невозможно (например, пытаемся превратить Кота в Собаку), `dynamic_cast` вернет `nullptr` (для указателей) или выбросит исключение (для ссылок).

Условие: Чтобы `dynamic_cast` работал, в базовом классе должна быть хотя бы одна виртуальная функция (обычно это деструктор).

13. Виртуальные функции. Сравнение с неvirtуальными. `virtual`, `final`, `override`.

Виртуальная функция — это метод, объявленный в базовом классе с ключевым словом `virtual`, который может быть переопределен в наследниках.

Ключевые слова: `virtual`, `override`, `final`

`virtual`: Ставится в базовом классе. Говорит компилятору: «Связывай эту функцию не по типу указателя, а по типу объекта в памяти».

`override` (C++11): Ставится в наследнике. Это проверка для компилятора. Если ты случайно ошибешься в имени или параметрах (например, `void draw(int)` вместо `void draw()`), компилятор выдаст ошибку, а не просто создаст новую функцию.

`final` (C++11): Ставится после `override` или у самого класса. Запрещает дальнейшее переопределение функции или наследование от класса. Полезно для безопасности и небольшого выигрыша в скорости.

2. Сравнение виртуальных и неvirtуальных функций

Неvirtуальные функции (Статическое связывание)

Компилятор смотрит на тип указателя. Если указатель `Animal*`, то всегда вызовется метод из `Animal`, даже если там лежит `Dog`. Решение принимается в момент сборки программы.

Виртуальные функции (Динамическое связывание)

Программа смотрит на тип объекта. Она заглядывает в таблицу виртуальных методов (VTable) и находит там адрес функции того класса, чей объект создан на самом деле. Решение принимается в момент выполнения.

Неvirtуальная функция вызывается исходя из того, какой тип переменной мы написали в коде (статическое связывание).

Виртуальная функция вызывается исходя из того, какой реальный объект лежит в памяти (динамическое связывание).

Для чего это нужно? Без виртуальных функций полиморфизм не работает. Если мы создадим список `Animal*` и положим туда собак, кошек и птиц, то

без virtual они все будут «просто животными» и не смогут лаять или мяукать по отдельности.

14. Таблица виртуальных функций. Как она используется в dynamic_cast.

1. Коротко: Что такое VTable (повторение)

Как мы уже обсуждали, если в классе есть хотя бы одна виртуальная функция, компилятор создает:

VTable (Virtual Table): таблицу (массив) с адресами виртуальных функций этого класса.

vptr: скрытый указатель внутри каждого объекта, который указывает на VTable его класса.

2. Связь VTable и RTTI

Для работы dynamic_cast одной таблицы функций мало. Нужно знать, что это за класс на самом деле. Поэтому компилятор добавляет в память RTTI (Run-Time Type Information).

Как это устроено: Обычно в самом начале (или перед началом) каждой таблицы VTable лежит специальный указатель на структуру type_info.

type_info — это «паспорт» класса. В нем хранится имя класса и информация о его родителях (иерархия наследования).

Важный вывод для зачета: dynamic_cast работает только с полиморфными типами (где есть хотя бы одна виртуальная функция). Почему? Потому что если нет виртуальных функций → нет VTable → нет ссылки на «паспорт» (RTTI) → негде проверить тип.

3. Как работает dynamic_cast (пошагово)

Когда ты пишешь `Base* ptr = dynamic_cast<Derived*>(basePtr);`, происходит следующее:

Проверка на null: Если `basePtr` равен `nullptr`, то и результат будет `nullptr`.

Переход по vptr: Программа заходит в объект, берет его vptr и прыгает в VTable.

Доступ к RTTI: Из VTable программа достает указатель на информацию о типе (type_info). Теперь она точно знает: «Ага, в памяти сейчас лежит объект класса Cat».

Проверка иерархии:

Программа смотрит, является ли реальный тип (Cat) тем, к которому мы приводим (Derived), или его наследником.

Она «пробегаёт» по дереву наследования, которое записано в структурах RTTI.

Результат:

Если путь в дереве найден (преобразование законно) → возвращается адрес объекта, приведенный к нужному типу.

Если путь не найден (например, пытаемся превратить Кота в Собаку) → возвращается nullptr (или выбрасывается исключение std::bad_cast, если работаем со ссылками).

15. Чистые виртуальные функции. Абстрактные и конкретные классы. Понятие интерфейса.

1. Чистые виртуальные функции (Pure Virtual Functions)

Чистая виртуальная функция — это функция, которая не имеет реализации в базовом классе. Она лишь объявляет «контракт»: любой класс, который хочет быть полноценным наследником, обязан реализовать эту функцию сам.

virtual void sound() = 0; // "= 0" делает функцию чистой виртуальной

2. Абстрактные и конкретные классы

Здесь всё просто: это деление классов по возможности создавать их объекты.

Абстрактный класс

Если в классе есть хотя бы одна чистая виртуальная функция, он автоматически становится абстрактным.

Главное правило: Нельзя создать объект абстрактного класса.

Цель: Быть базой (фундаментом) для других классов.

Конкретный класс

Это класс, в котором все чистые виртуальные функции родителя были переопределены и получили реализацию.

Главное правило: Можно создавать объекты этого класса.

3. Понятие интерфейса

Интерфейс — это класс, который:

Содержит только чистые виртуальные функции.

Не имеет никаких полей данных (переменных).

Имеет виртуальный деструктор.

16. Виртуальный деструктор. Чистый виртуальный деструктор.

1. Виртуальный деструктор

Зачем он нужен? Когда ты удаляешь объект через указатель на базовый класс (`Base* ptr = new Derived(); delete ptr;`), компилятор смотрит на тип указателя. Если деструктор не виртуальный, вызовется только деструктор `Base`. Все ресурсы, которые выделил класс `Derived`, останутся в памяти — это утечка.

Как это работает? Ключевое слово `virtual` у деструктора заставляет программу использовать таблицу виртуальных методов (`VTable`).

Сначала вызывается деструктор самого «младшего» класса (`Derived`).

Затем автоматически по цепочке вызываются деструкторы всех родителей.

2. Чистый виртуальный деструктор (Pure Virtual Destructor)

Иногда бывает ситуация: тебе нужно сделать класс абстрактным (чтобы нельзя было создавать его объекты), но у тебя нет для него подходящей «чистой» функции. Все методы уже имеют какую-то реализацию.

В отличие от обычных чистых виртуальных функций, для чисто виртуального деструктора ОБЯЗАТЕЛЬНО нужно написать реализацию (тело функции) вне класса.

Почему? Потому что при уничтожении любого наследника (`Derived`), его деструктор в конце своей работы обязан вызвать деструктор базового класса. Если тела у `~AbstractBase()` не будет, линкер выдаст ошибку, так как ему нечего будет вызывать.

17. Вложенные классы. Агрегирование и композиция.

1. Вложенные классы (Nested Classes)

Вложенный класс — это класс, описанный внутри другого класса.

Доступ: В современном C++ (начиная с C++11) вложенный класс считается «членом» внешнего класса, поэтому он имеет доступ ко всем `private` и `protected` членам внешнего класса. И наоборот: внешний класс имеет доступ к закрытым членам вложенного.

Связь объектов: Важно понимать: создание объекта внешнего класса не означает автоматическое создание объекта вложенного. Это просто описание типа внутри типа.

Область видимости: Вложенный класс находится внутри области видимости (scope) внешнего класса. Чтобы создать его объект снаружи, нужно использовать оператор ::

2. Агрегирование и композиция

Обе эти концепции описывают отношение «имеет» (has-a) или «является частью» (part-of). Разница заключается в силе связи и времени жизни объектов.

Композиция (Strong Ownership)

Это жесткая связь. Объект-часть не может существовать без объекта-целого. Если мы уничтожаем «целое», «части» уничтожаются автоматически.

Пример: Машина и её Двигатель. Если машина идет под пресс, двигатель уничтожается вместе с ней.

В коде: Часть обычно является полем класса (не указателем).

1. Композиция: «Часть не живет без целого»

При композиции: деструктор «целого» автоматически тянет за собой деструкторы всех «частей». Программисту не нужно об этом думать.

При агрегировании: деструктор «целого» удаляет только сам указатель (просто переменную с адресом), но не трогает объект, на который этот адрес указывал. Объект-часть остается в памяти, пока его не удалит тот, кто его создал.

Агрегирование (Weak Ownership)

Это слабая связь. Объект-часть может существовать сам по себе и может принадлежать разным «целым». Если «целое» уничтожается, «часть» выживает.

Пример: Университет и Профессор. Если университет закроют, профессор просто пойдет работать в другое место.

В коде: Реализуется через указатели или ссылки.

2. При АГРЕГИРОВАНИИ (Слабая связь)

Правило: «Главный ушел, зависимый остался».

Здесь основной объект хранит только адрес (указатель или ссылку) на другой объект, который был создан где-то в другом месте.

Что происходит: Когда вызывается деструктор основного класса (например, Университет), удаляется только сам указатель (переменная, хранящая адрес). Но тот объект, на который он указывал (Студент), не трогается.

Кто удаляет: Тот, кто создавал этот объект изначально (внешний код). Сам класс-владелец агрегации не имеет права удалять этот объект в своем деструкторе.

Итог: После удаления «целого», «часть» продолжает преспокойно жить в памяти.

18. Множественное наследование. Проблема ромбовидного наследования.

1. Что такое множественное наследование?

Это ситуация, когда один класс наследуется сразу от двух и более базовых классов.

```
class PowerBank { ... };
```

```
class Flashlight { ... };
```

```
// Класс-фонарик с функцией зарядки наследует обоих
```

```
class SmartFlashlight : public PowerBank, public Flashlight { ... };
```

Особенности:

Объект SmartFlashlight будет содержать в себе поля и методы обоих родителей.

Конструкторы вызываются в том порядке, в котором классы перечислены при объявлении (сначала PowerBank, потом Flashlight).

19. Виртуальный родительский класс. Конструкторы и деструкторы при множественном наследовании.

2. Конструкторы при множественном наследовании

Здесь есть строгое правило порядка. Когда создается объект Child, конструкторы вызываются в следующем порядке:

Виртуальные базовые классы (в порядке их объявления в списке наследования).

Невиртуальные базовые классы (в порядке их объявления в списке наследования слева направо).

Поля-объекты класса (члены данных).

Сам конструктор наследника.

3. Деструкторы при множественном наследовании

Тут всё просто: работает правило «обратного порядка».

Порядок вызова деструкторов:

Сам деструктор наследника.

Поля-объекты класса (в порядке, обратном объявлению).

Невиртуальные базовые классы (справа налево по списку наследования).

Виртуальные базовые классы.

Запомни: Виртуальный родитель — это «фундамент фундамента». Он закладывается первым, а сносится самым последним.

20. Аварийные ситуации. Объекты-исключения. `try`, `catch`, `throw`.

1. Механизм Try, Catch, Throw

Это «три кита», на которых держится обработка исключений:

`throw` (бросить): Используется, когда программа обнаруживает ошибку, которую не может решить на месте. Мы «бросаем» объект-исключение.

`try` (попробовать): Блок кода, в котором мы ожидаем возможные ошибки. Программа «наблюдает» за этим участком.

`catch` (поймать): Блок кода, который выполняется только если внутри `try` произошло исключение. Здесь мы описываем, что делать с ошибкой.

2. Объекты-исключения

В C++ можно «бросить» практически что угодно: число, строку или даже кастомный объект. Но по правилам хорошего тона (и для удобства) принято бросать объекты классов, унаследованных от `std::exception`.

Стандартная иерархия (библиотека `<stdexcept>`):

`std::runtime_error` — ошибки, которые можно обнаружить только во время работы (например, деление на ноль).

`std::logic_error` — ошибки в логике программы (например, передача неверного аргумента).

`std::out_of_range` — выход за пределы массива.

21. Оператор `typeid`. Класс `type_info`. Операторы приведения типа в стиле C++: `const_cast`, `dynamic_cast`, `static_cast`, `reinterpret_cast`.

Это один из самых «технических» вопросов. Здесь проверяется твое умение не просто писать код, а управлять типами данных осознанно и безопасно.

1. RTTI: Оператор typeid и класс typeid

RTTI (Run-Time Type Information) — это механизм, позволяющий программе определять тип объекта прямо во время выполнения.

Оператор typeid

Он позволяет получить информацию о типе выражения или объекта.

Возвращает ссылку на константный объект класса typeid.

Если тип объекта полиморфный (есть виртуальные функции), typeid заглянет в VTable и определит реальный тип объекта, а не тип указателя.

Класс typeid (библиотека <typeid>)

Этот класс хранит информацию о типе. У него есть два главных метода:

.name() — возвращает строку с именем типа (зависит от компилятора, может быть сокращенным).

Операторы == и != — позволяют сравнивать типы между собой.

2. Операторы приведения типов (C++ Casts)

В C++ принято отказываться от старого стиля приведения (int)x, так как он слишком «грубый» и незаметный в коде. Вместо него используют 4 специальных оператора.

1. static_cast (Статическое приведение)

Самый часто используемый. Проверяется при компиляции.

Для чего: Приведение родственных типов (int в double, float в int), Upcasting (от наследника к родителю).

Риск: Не проверяет безопасность при Downcasting (от родителя к ребенку). Если там не тот объект — программа поведет себя непредсказуемо.

2. dynamic_cast (Динамическое приведение)

Использует RTTI. Проверяется во время выполнения.

Для чего: Безопасный Downcasting по иерархии наследования.

Как работает: Если приведение невозможно, вернет nullptr (для указателей) или выбросит исключение (для ссылок).

Условие: Требуется наличия хотя бы одной виртуальной функции в базовом классе.

3. const_cast (Снятие константности)

Единственный оператор, который может «убрать» или «добавить» const или volatile.

Для чего: Работа со старыми API, которые принимают char*, хотя не меняют строку.

Риск: Изменение изначально константного объекта через const_cast — это неопределенное поведение (UB).

4. reinterpret_cast (Переинтерпретация)

Самый опасный cast. Он просто заставляет компилятор трактовать последовательность битов как другой тип.

Для чего: Превращение указателя в число (long) и обратно, работа с бинарными данными, системное программирование.

Риск: Никаких проверок. Вся ответственность на программисте.